DESIGN, BUILD, SHIP · MPCS 51238 · SPRING 2026

Week 4: Build & Deploy Complex Systems

Background workers, Railway, Supabase Realtime, upsert, monorepos, and multi-model fluency

> **What this document is:** A companion to the Week 4 slides and class session. It covers the multi-service system we built in class — a background worker on Railway polling ESPN's API, Supabase for database + auth + Realtime subscriptions, and a Next.js frontend on Vercel displaying live scores. This is the reference you come back to when you're building. Think of it as the textbook chapter for this week.

By the end of this week, you should understand:

- What makes a “system” different from an “app” — multiple services, multiple platforms
- What a background worker is and why it runs independently of users
- How Railway deploys long-running processes via GitHub
- How Supabase Realtime pushes data to connected clients via WebSockets
- Why upsert exists and when to use it instead of insert
- The difference between service role key and anon key in the context of a worker
- How to work with an unofficial API like ESPN's scoreboard endpoint
- How to structure a monorepo with multiple services deploying to different platforms
- What Codex CLI is and why multi-model fluency matters

1 Systems Architecture: From Apps to Systems

You've built apps. Now you're building something with independent moving parts.

In Week 3 you built a full-stack application: a Next.js frontend on Vercel talking to a Supabase database. That's one service, deployed to two platforms. The data flows in a straight line — browser to server to database and back.

This week you're building a **system**. The difference? A system has **multiple independent services** that communicate through shared infrastructure. Your live sports dashboard has three services running on three platforms, and they don't all know about each other:

ESPN API → Railway Worker → Supabase → Vercel Frontend

The worker doesn't know about the frontend. The frontend doesn't know about the worker. They're connected through the database — the worker writes to it, the frontend reads from it. This is **loose coupling**, and it's the fundamental pattern of distributed systems.

The Four Places Code Can Run

In a modern web system, your code doesn't all live in one place. It runs in four distinct environments, each with different capabilities and constraints:

1. Client (Browser)

React components, user interactions, local state. Runs on the user's device. Can't access secrets, can't make direct server-to-server calls. Subject to CORS.

In our system: the Next.js frontend showing scores.

2. Server (API Routes)

Next.js API routes, server components. Runs on Vercel's servers. Can hold secrets, call external APIs, shape responses before sending them to the client.

In our system: authentication flows, favorite team queries.

3. Background Worker

A long-running process with no user-facing interface. Runs continuously on its own schedule. No HTTP requests trigger it — it triggers itself.

In our system: the Railway worker polling ESPN every 30 seconds.

4. Database

Supabase (Postgres). Stores data, enforces RLS, pushes Realtime updates. The shared state that connects all other services. Also runs functions (triggers, RPC).

In our system: game scores, user accounts, favorites, Realtime subscriptions.

Why Separation of Concerns Matters

You could build the entire scoreboard as one monolithic app — a single Next.js server that polls ESPN, stores data, and serves the frontend. It would work. But here's what goes wrong at scale:

- **If the frontend crashes, the worker stops.** With separate services, the worker keeps polling ESPN even if the website is down. Your data stays fresh.
- **If you need to redeploy the UI, you interrupt polling.** Independent services mean you can deploy the frontend without touching the worker.
- **Different runtime needs.** The frontend needs to respond to HTTP requests fast. The worker needs to run a loop forever. These are fundamentally different tasks that benefit from different environments.
- **Different scaling needs.** If 10,000 users hit your frontend, you scale the frontend. The worker still only needs one instance — it's still making one API call every 30 seconds.

> **The industry pattern:** Every complex application you use works like this. Uber has background services tracking driver locations. Instagram has workers processing image uploads. Slack has services managing real-time message delivery. The specific tools differ, but the shape is always the same: separate services, shared database, independent deployment.

The Week-by-Week Progression

Week	What you built	Architecture
1	Static site	1 HTML file → Vercel
2	Interactive app	Next.js → Vercel
3	Full-stack app	Next.js → Vercel → Supabase (2 platforms)
4	**System**	**Next.js → Vercel + Railway (worker) + Supabase (3 platforms)**

Each week adds a layer. This week is the biggest jump: from a single application to a distributed system with independent services.

Check Your Understanding

1. What's the key difference between an “app” (Week 3) and a “system” (Week 4)? Why can't a single Next.js server handle everything the scoreboard needs?

   Think about what happens when you redeploy: with one server, polling stops. With separate services, the worker keeps running independently of the frontend.

2. Why doesn't the frontend talk directly to ESPN? Why does it go through Supabase instead?

   500 users means 500 redundant API calls. The worker fetches once, writes to Supabase, and Supabase serves everyone. Plus CORS blocks browser-to-ESPN requests.

2 Background Workers: Code That Runs Without Users

A tiny robot that does work while everyone's asleep.

Every application you've built so far is **request-driven**: a user does something (clicks a button, loads a page), and your code responds. A background worker flips this model. It runs **continuously**, on its own schedule, regardless of whether anyone is using your app. No HTTP request triggers it. No user starts it. It just runs.

What a Background Worker Does

A worker is a process — a running program — that performs tasks on a schedule or in response to events. In our system, the worker follows the **polling pattern**:

Fetch data → Parse response → Upsert to database → Sleep 30 seconds → Repeat

That's the entire loop. The worker wakes up, checks ESPN for the latest scores, writes what it finds to Supabase, and goes back to sleep. It doesn't know about your frontend. It doesn't know about your users. It just does its job, over and over, forever.

Why Workers Solve the Scaling Problem

Consider the naive approach: every user's browser calls ESPN directly every few seconds.

```
// The naive approach: every browser polls ESPN
User 1's Browser → ESPN API → Display scores
User 2's Browser → ESPN API → Display scores
User 3's Browser → ESPN API → Display scores
  (× 500 users = 500 calls every few seconds)
```

This falls apart for multiple reasons:

- **Scale:** 500 users at 5-second intervals = 6,000 API calls per minute. You'll get rate-limited or blocked.
- **Redundancy:** The Lakers score is the same for everyone. Why fetch it 500 times?
- **CORS:** ESPN's API doesn't send CORS headers — the browser can't call it directly anyway.
- **No persistence:** Refresh the page and completed games vanish. There's no history.

With a worker, **one process makes one API call every 30 seconds**, regardless of how many users are connected. The database stores the results. Realtime pushes them to everyone. 1 fetch serves 500 users.

The Mental Model

> *Think of the worker as a tiny robot. It wakes up every 30 seconds, walks over to ESPN, checks the latest scores, walks back to your database, writes down what it found, and goes back to sleep. It doesn't care if anyone is watching. It doesn't care if it's 3 AM. It just does its job.*

This mental model extends beyond sports scores. Workers handle email queues, image processing, data synchronization, scheduled reports, cleanup tasks — anything that needs to happen on a schedule or continuously, without a user triggering it.

The Worker Code

The worker is a simple Node.js script. No web framework, no HTTP server — just a loop:

```
import { createClient } from '@supabase/supabase-js'

const supabase = createClient(
  process.env.SUPABASE_URL,
  process.env.SUPABASE_SERVICE_ROLE_KEY  // not the anon key!
)

async function fetchAndUpdate() {
  // 1. Fetch from ESPN
  const res = await fetch('https://site.api.espn.com/apis/site/v2/sports/baske
  const data = await res.json()

  // 2. Parse the response into our schema
  const games = data.events.map(event => ({
    id: event.id,
    home_team: event.competitions[0].competitors[0].team.displayName,
    away_team: event.competitions[0].competitors[1].team.displayName,
    home_score: parseInt(event.competitions[0].competitors[0].score),
    away_score: parseInt(event.competitions[0].competitors[1].score),
    status: mapStatus(event.status.type.name),
    game_clock: event.status.type.shortDetail,
    // ... more fields
  }))

  // 3. Upsert to Supabase
  await supabase.from('games').upsert(games, { onConflict: 'id' })
}

// Run immediately, then every 30 seconds
fetchAndUpdate()
setInterval(fetchAndUpdate, 30000)
```

Check Your Understanding

1. What would happen if you removed the worker and had each user's browser poll ESPN directly? List at least three problems.

   CORS blocks it, 500 users = 500 redundant calls (rate limiting), no persistence across refreshes, and no shared data layer.

2. Why does the worker use `setInterval` instead of responding to HTTP requests? How is this fundamentally different from an API route?

   An API route is reactive — it waits for a request. The worker is proactive — it runs on its own schedule. Nobody calls the worker; it calls itself.

3 Railway: Deploying Background Services

Vercel runs web servers. Railway runs anything.

Vercel is designed for web applications — things that respond to HTTP requests. But a background worker doesn't serve web pages. It doesn't respond to requests at all. It's a long-running Node.js process that loops forever. Vercel can't run that. **Railway can.**

What Is Railway?

Railway is a cloud platform for deploying **any process**. Web servers, background workers, cron jobs, databases, bots — if it runs, Railway can host it. Think of it as a simpler alternative to AWS or Google Cloud, designed for developers who want to deploy without infrastructure complexity.

Vercel

- Optimized for web frameworks (Next.js, React)
- Serverless functions with request/response model
- Auto-scales based on traffic
- Functions have time limits (10s–300s)

Perfect for: your frontend and API routes.

Railway

- Runs any process — no framework required
- Long-running processes (runs forever)
- Persistent connections, background loops
- No time limits on execution

Perfect for: your background worker.

Deploying via GitHub

Railway connects to your GitHub repository. When you push code, Railway automatically rebuilds and redeploys your service. The deploy flow:

Push to GitHub → Railway detects change → Builds your code → Deploys new version → Worker starts running

For a monorepo, you configure Railway to watch a specific directory (e.g., `apps/worker`). Changes to `apps/web` won't trigger a Railway redeploy, and vice versa.

Environment Variables on Railway

Same concept as Vercel: add your secrets through the Railway dashboard, not in your code. For the worker, you need two variables:

Variable	Value	Why
`SUPABASE_URL`	`https://xxx.supabase.co`	Where to connect to the database
`SUPABASE_SERVICE_ROLE_KEY`	`sb_secret_...`	Bypasses RLS — worker writes data for all users

No `NEXT_PUBLIC_` prefix needed — Railway doesn't have a browser bundle. All variables are server-side by default.

The $5 Trial Tier

Railway offers a trial plan with $5 of free credits. No credit card required. This is more than enough for the entire assignment — a simple Node.js worker running one fetch every 30 seconds uses minimal resources. You'll likely spend less than $1 over the whole week.

Railway MCP

Railway has an MCP server that lets Claude check your deployment status and read logs directly. Once connected, you can ask Claude things like “Is my worker running?” or “Show me the Railway logs” and Claude can answer by querying Railway's API.

Check Your Understanding

1. Why can't you deploy the background worker to Vercel? What limitation of serverless functions makes Railway necessary?

   Vercel functions have time limits and are request-driven. The worker needs to run a loop forever — it's not responding to HTTP requests, it's polling on its own schedule.

2. What happens when you push code to GitHub with Railway connected? How is this similar to Vercel's deployment model?

   Both auto-deploy on push. The difference is what they deploy: Vercel deploys a web app, Railway deploys any process. Same trigger (git push), different runtime environments.

4 Supabase Realtime: Push, Don't Pull

Instead of asking for updates every few seconds, the database tells you when something changes.

In the naive approach, the frontend would poll the database repeatedly: “Any new scores? Any new scores? Any new scores?” every few seconds. This works but it's wasteful — most of those requests return “nope, nothing changed.” Supabase Realtime flips the model: **the database pushes updates to you when data changes**.

Polling (Pull) vs. Subscriptions (Push)

Polling (Pull)

The client asks the server for data at regular intervals. Most requests are wasted because nothing has changed.

```
// Poll every 5 seconds
setInterval(async () => {
  const { data } = await supabase
    .from('games').select('*')
  setGames(data)
}, 5000)
```

Simple but wasteful. 500 users = 6,000 queries/min to your database.

Subscription (Push)

The client opens a persistent connection. The server sends data only when something changes. Zero wasted requests.

```
// Subscribe to changes
supabase.channel('games')
  .on('postgres_changes',
    { event: '*',
      schema: 'public',
      table: 'games' },
    payload => {
      updateGame(payload.new)
    })
  .subscribe()
```

Efficient and instant. 500 users = 1 push from the database.

How Supabase Realtime Works

Under the hood, Supabase Realtime uses **WebSockets** — persistent, two-way connections between the browser and the server. Unlike HTTP (which is request-response, one shot at a time), a WebSocket stays open. Data can flow in either direction at any time.

When you call `.subscribe()`, your browser opens a WebSocket connection to Supabase. Supabase watches the Postgres replication log — a built-in Postgres feature that records every change to the database. When a row changes in the `games` table, Supabase sees it in the replication log and pushes the change through the WebSocket to every connected client.

Enabling Realtime on a Table

Realtime is **not enabled by default**. You have to opt in for each table. In the Supabase dashboard:

1. Go to **Database** → **Replication**
2. Find the `games` table
3. Toggle Realtime **on**

If you forget this step, your subscribe call will connect but never fire. This is one of the most common “it's not working” bugs — everything looks right in the code, but Replication isn't enabled.

The Subscribe Pattern

Here's the full pattern you'll use in your React component:

```
import { useEffect, useState } from 'react'

function Scoreboard() {
  const [games, setGames] = useState([])

  useEffect(() => {
    // 1. Initial fetch — get current state
    supabase.from('games').select('*').then(({ data }) => setGames(data))

    // 2. Subscribe — get future changes
    const channel = supabase
      .channel('games')
      .on('postgres_changes',
        { event: '*', schema: 'public', table: 'games' },
        payload => {
          // Update the specific game that changed
          setGames(prev =>
            prev.map(g => g.id === payload.new.id ? payload.new : g)
          )
        })
      .subscribe()

    // 3. Cleanup — unsubscribe when component unmounts
    return () => { supabase.removeChannel(channel) }
  }, [])

  return /* render games */
}
```

The pattern has three parts: **initial fetch** (get the current state of all games), **subscribe** (listen for any future changes), and **cleanup** (unsubscribe when the component unmounts to avoid memory leaks).

> **The data flow in full:** The worker upserts a score change to Supabase. Supabase detects the change in the Postgres replication log. Supabase pushes the new row through the WebSocket to every connected browser. The React callback updates local state. The UI re-renders. The user sees the score change — instantly, without refreshing, without polling.

Check Your Understanding

1. Why is push (Realtime) more efficient than pull (polling) when 500 users are watching the same scoreboard?
   Polling: 500 users each query the database every 5 seconds = 6,000 queries/min, most returning no changes. Push: the database sends one update when data actually changes, delivered to all 500 connections simultaneously.
2. You set up Realtime subscriptions in your code, but your frontend never receives updates even though the worker is writing data. What did you probably forget?
   Realtime replication isn't enabled by default. You need to toggle it on for the specific table in Supabase Dashboard → Database → Replication.

5 Upsert: Create or Update

The same game gets updated dozens of times. You need an operation that handles both cases.

The worker polls ESPN every 30 seconds. The first time it sees a game, it needs to **create** a new row. Every subsequent time, it needs to **update** the existing row with the latest score. You could write logic to check "does this game exist?" before deciding whether to insert or update. Or you can use **upsert**, which does both in one operation.

How Upsert Works

`upsert` = **insert** + **update**. It tries to insert the row. If a row with the same primary key (or specified conflict column) already exists, it updates that row instead. No conditional logic needed.

```
// Without upsert — you'd have to check first
const { data: existing } = await supabase
  .from('games').select('id').eq('id', gameId)

if (existing.length > 0) {
  await supabase.from('games').update(gameData).eq('id', gameId)
} else {
  await supabase.from('games').insert(gameData)
}

// With upsert — one operation, same result
await supabase.from('games').upsert(games, { onConflict: 'id' })
```

The `onConflict` Parameter

The `onConflict` tells Supabase which column to check for existing rows. In our case, `id` is the ESPN game ID. If a row with that ID already exists, update it. If not, insert a new one. This is the **conflict resolution strategy** — the database knows exactly what makes a row "the same game."

You can also upsert arrays — pass an entire batch of games at once, and Supabase handles each one individually:

```
// Upsert all games at once — each resolved independently
const games = data.events.map(event => ({
  id: event.id,
  home_team: /* ... */,
  home_score: /* ... */,
  status: /* ... */,
}))

await supabase.from('games').upsert(games, { onConflict: 'id' })
```

> **Why not just delete and reinsert?** You could delete all games and insert fresh ones every 30 seconds. But that would trigger Realtime events for every row, even unchanged ones. Upsert only triggers updates for rows that actually changed, making Realtime efficient and accurate.

Check Your Understanding

1. Why does the worker use upsert instead of insert? What would go wrong if it used insert every time it polled ESPN?
   Insert would fail with a duplicate key error the second time the worker polls, because the game row already exists. Upsert handles both the first-seen (insert) and subsequent (update) cases automatically.

6 Service Role Key vs. Anon Key

Week 3 introduced the concept. This week you see why it matters in a multi-service system.

In Week 3, you learned that Supabase gives you two keys: a publishable (anon) key that respects Row-Level Security, and a secret (service role) key that bypasses it. This week, the worker makes the distinction concrete — you're forced to choose which key to use in a new context.

Two Keys, Two Contexts

Anon Key (Frontend)

- Used by the Next.js frontend
- Respects RLS — users only see their own favorites
- Paired with a user's auth token (Clerk/Supabase Auth)
- Safe to expose in the browser

Context: a specific user is making a request. RLS knows who they are.

Service Role Key (Worker)

- Used by the Railway background worker
- Bypasses RLS — can read/write all data
- No user context — the worker is infrastructure, not a person
- Must stay on the server, never in client code

Context: trusted infrastructure writing data for everyone.

Why the Worker Needs Service Role

The worker isn't a user. It's not logged in. It doesn't have a session token or a user ID. It's a background process that writes game scores that *all* users will see. If you used the anon key, RLS policies would block the writes — the worker has no identity for RLS to check against. The service role key says: “I'm trusted infrastructure. Let me write.”

What Goes Wrong with the Wrong Key

Scenario	What happens
Worker uses **anon key**	RLS blocks the upsert. No user identity means no rows can be written. The worker fails silently or returns empty results. Your scoreboard stays empty.
Frontend uses **service role key**	RLS is bypassed. *Any user can read all data from every table, including other users' favorites, auth data, and anything else in the database.* This is a critical security vulnerability.

> **The rule:** The service role key is for **trusted code that runs on servers you control**. The anon key is for **code that runs in the browser**. The worker is server-side infrastructure → service role key. The frontend ships to users' browsers → anon key. Never the reverse.

Environment Variables by Platform

Platform	Variable	Key Type
Vercel (frontend)	`NEXT_PUBLIC_SUPABASE_ANON_KEY`	Anon — respects RLS
Railway (worker)	`SUPABASE_SERVICE_ROLE_KEY`	Service role — bypasses RLS

Notice the frontend key has `NEXT_PUBLIC_` because it needs to be in the browser. The worker key has no prefix because it's server-only.

Check Your Understanding

1. Why would the worker's upsert fail if you used the anon key instead of the service role key?
   The anon key respects RLS. The worker has no user identity (no auth token, no session), so RLS policies that check for a user will deny the write. The service role key bypasses RLS entirely.
2. A student puts the service role key in a `NEXT_PUBLIC_` environment variable and uses it in the frontend "because it has more access." What's the worst-case scenario?
   Anyone can open DevTools, find the key, and use it to read, modify, or delete every row in every table — bypassing all RLS policies. It's a full database compromise.

7 The ESPN API: Working with Unofficial APIs

No documentation, no SLA, no API key. Just a URL that returns live sports data.

The ESPN scoreboard endpoint we use is an **unofficial API**. ESPN built it for their own website — when you load espn.com/nba/scoreboard, their JavaScript calls this endpoint to get the data. We're using the same endpoint. There's no official documentation, no developer portal, no API key. It just works — until ESPN decides to change it.

The Endpoint

```
https://site.api.espn.com/apis/site/v2/sports/basketball/nba/scoreboard
```

Paste that URL into your browser. You'll get a JSON response with every NBA game for today. No authentication required. The response structure is deeply nested but consistent.

Key Response Fields

The response contains an `events` array. Each event is a game. Here's what matters:

Path	What it contains	Example
`events[].id`	Unique game ID	`"401584701"`
`events[].competitions[0].competitors[]`	Home and away teams	Array of two team objects
`competitors[].team.displayName`	Full team name	`"Los Angeles Lakers"`
`competitors[].team.abbreviation`	Short code	`"LAL"`
`competitors[].score`	Current score (string)	`"108"`
`competitors[].team.logo`	Team logo URL	ESPN CDN URL
`events[].status.type.name`	Game status	`"STATUS_SCHEDULED"`, `"STATUS_IN_PROGRESS"`, `"STATUS_FINAL"`
`events[].status.type.shortDetail`	Human-readable status	`"Q3 4:32"`, `"Final"`, `"7:30 PM ET"`

Mapping ESPN Status to Our Schema

```
function mapStatus(espnStatus) {
  switch (espnStatus) {
    case 'STATUS_SCHEDULED':  return 'pre'
    case 'STATUS_IN_PROGRESS': return 'in'
    case 'STATUS_FINAL':       return 'post'
    default:                    return 'pre'
  }
}
```

CORS: Why the Browser Can't Call It

If you try to fetch this URL from a React component in the browser, you'll get a CORS error. ESPN's server doesn't include `Access-Control-Allow-Origin` headers in its response, which means browsers will block the request. This is a security feature of browsers, not a bug.

But the worker runs on a server (Railway), not in a browser. **Server-to-server requests don't have CORS restrictions.** CORS is a browser-only policy. Your Railway worker can call ESPN's API freely.

> **This is another reason the architecture uses a background worker** — even if you wanted every user to fetch directly from ESPN, their browsers literally can't. The worker solves the CORS problem and the scaling problem simultaneously.

Polling Etiquette

Even though ESPN's API is free and keyless, you should be a responsible consumer:

- **30-second intervals** — scores don't change faster than this. Polling more aggressively wastes resources and risks getting blocked.
- **Error handling** — the API might be down temporarily. Catch errors, log them, and keep running. Don't crash the worker on a single failed fetch.
- **Backoff on errors** — if you get repeated failures, wait longer between retries. Don't hammer a server that's already struggling.
- **One instance** — run one worker, not ten. One fetch every 30 seconds is polite. Ten fetches every 30 seconds is not.

```
async function fetchAndUpdate() {
  try {
    const res = await fetch(ESPN_URL)
    if (!res.ok) throw new Error(`ESPN returned ${res.status}`)
    const data = await res.json()
    // ... parse and upsert
    console.log(`Updated ${data.events.length} games`)
  } catch (err) {
    console.error('Fetch failed:', err.message)
    // Don't crash — the next interval will retry
  }
}
```

Check Your Understanding

1. What does “unofficial API” mean? What risks come with building on an API that has no documentation and no SLA?

   It means ESPN built it for their own use and doesn't guarantee it for third parties. They could change the response structure, add rate limits, or shut it down without notice. Your system should handle failures gracefully.

2. Why does the browser get a CORS error when calling ESPN's API, but the Railway worker doesn't? What is CORS actually protecting?

   CORS is a browser-only security policy that prevents a webpage on your domain from reading responses from a different domain. Servers don't enforce CORS — it's a browser sandbox rule. The worker runs on a server, so it's not subject to CORS.

8 Monorepo: One Repo, Multiple Services

Your frontend and your worker are different services. They live in the same repository.

A **monorepo** is a single Git repository that contains multiple projects or services. Instead of having one repo for the frontend and another for the worker, everything lives together:

```
nba-scoreboard/
  CLAUDE.md            # Describes the whole system
  apps/
    web/              # Next.js frontend → deploys to Vercel
      package.json
      app/
      ...
    worker/           # Node.js worker → deploys to Railway
      package.json
      index.js
      ...
```

Why One Repo?

- **Shared context.** Both services are part of the same system. One repo means one place for issues, one place for PRs, one history. Claude can see the whole system at once.
- **Atomic changes.** If you change the database schema, you might need to update both the worker and the frontend. In a monorepo, that's one commit.
- **Simpler CLAUDE.md.** One file describes the entire architecture — where each service lives, how they connect, what environment variables they need. Claude understands the full picture.

Each Service Deploys Independently

Even though the code lives in one repo, each service deploys to a different platform:

Directory	Platform	Trigger
`apps/web/`	Vercel	Git push (Vercel watches this directory)
`apps/worker/`	Railway	Git push (Railway watches this directory)

Both Vercel and Railway can be configured to watch specific directories in a monorepo. A change to `apps/web/` triggers a Vercel deploy but not a Railway deploy, and vice versa.

CLAUDE.md at the Root

The root-level `CLAUDE.md` is the single source of truth for the entire system. It describes the architecture, the data flow, which services exist, and how they

connect. When you open Claude Code at the repo root, Claude reads this file and immediately understands the full system — not just one service.

> **Monorepos are industry standard** for systems with tightly related services. Google famously runs a monorepo with billions of lines of code. Your two-service project is the simplest version of this pattern, but the principle scales.

Check Your Understanding

1. Why do the frontend and worker live in the same repository instead of separate ones? What advantage does a monorepo give you when working with Claude Code?
   One repo means Claude can see both services at once. Changes that affect both (like a schema change) can be one commit. The root CLAUDE.md describes the full system, giving Claude complete context.

9 Claude Code + Codex: Multi-Model Fluency

Same workflow, different instruments. The skill is the pattern, not the product.

You've spent three weeks building with Claude Code. This week you learn that the same workflow pattern exists in other tools — and that fluency in the pattern matters more than fluency in any one tool.

What Is Codex CLI?

Codex CLI is OpenAI's terminal-based AI coding agent — a direct analog to Claude Code. Install with `npm i -g @openai/codex`, run with `codex`. It reads files, edits code, runs commands, and iterates. Same concept, different model underneath (GPT instead of Claude).

AGENTS.md vs. CLAUDE.md

Both are the same concept: a project context file that tells the AI agent what your project is, how to build it, and what conventions to follow.

	CLAUDE.md	AGENTS.md
Read by	Claude Code	Codex, Copilot CLI, Gemini CLI, Cursor, and Claude Code
Created by	Anthropic	Open standard (Linux Foundation)
Format	Markdown	Markdown

Everything you've put in your `CLAUDE.md` — project description, build commands, conventions — that same information goes in `AGENTS.md` for other agents. The skill you've been building is transferable. It's worth having both files in your project.

Approval Modes

Codex has three modes, similar to Claude Code's permission system:

Auto (default)
Reads files, edits code, runs commands within the working directory. Asks before anything outside that scope or network access.

Read-only
Consultative mode. Browses files but won't make changes or run commands until you approve each one.

Full Access
Works across your machine including network access without asking. Use sparingly.

Why Multi-Model Fluency Matters

- **Different models have different strengths.** You'll discover which is better at what as you use them — that intuition is part of the skill.
- **The tools are converging.** Cursor, Claude Code, Codex — the interfaces are all approaching the same pattern. The transferable skill is understanding the workflow, not memorizing one tool's syntax.
- **Avoid lock-in.** Models and pricing change rapidly. Developers who can switch tools retain leverage.
- **The "second opinion" pattern.** You can use one agent to review another's work. Two models checking each other catches more bugs than either alone.

> **The core loop is identical:** open terminal → describe what you want → review changes → iterate. Whether you're in `claude` or `codex`, the rhythm is the same. By the end of this course, you'll be fluent in *agentic engineering*, not just one product.

Check Your Understanding

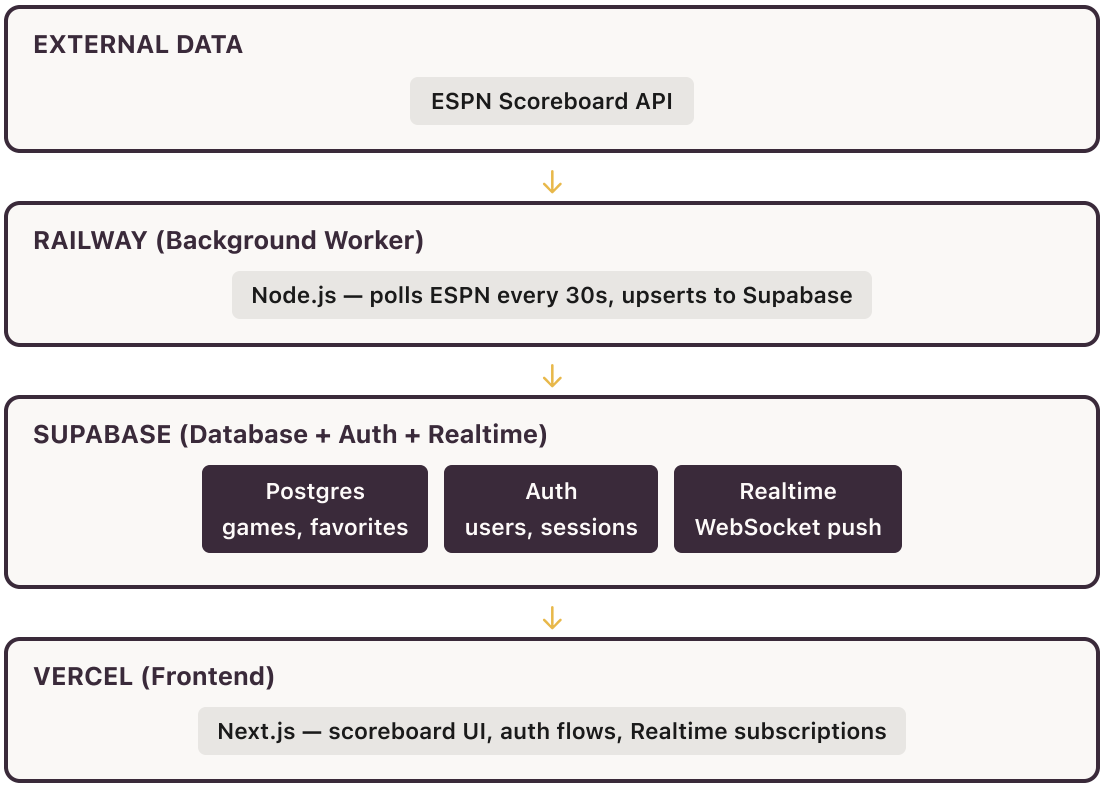
1. Why is it valuable to have both a `CLAUDE.md` and an `AGENTS.md` in your project? What does each file enable?

   CLAUDE.md is read by Claude Code. AGENTS.md is an open standard read by Codex, Copilot CLI, Gemini CLI, and others. Having both means any AI coding agent can understand your project immediately. The skill of writing good project context files transfers across tools.

10 Putting It All Together: The Architecture

Five services, three platforms, one data flow. Here's how the whole system works.

Full Architecture



Data Flow Walkthrough

Let's trace what happens when a score changes during a live game:

1. Score changes in the real world

LeBron hits a three-pointer. The Lakers go up 98–95. ESPN's internal system updates the score.

2. Worker polls ESPN

Your Railway worker's next 30-second interval fires. It fetches the ESPN scoreboard endpoint and gets the updated score in the JSON response.

3. Worker upserts to Supabase

The worker parses the response, maps the status, and calls `supabase.from('games').upsert(games, { onConflict: 'id' })`. The Lakers game row is updated with the new score. Because it uses the service role key, RLS doesn't block the write.

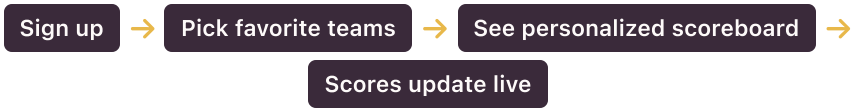
4. Supabase Realtime pushes the change

Supabase detects the row change in the Postgres replication log. It pushes the updated row through WebSocket connections to every browser subscribed to the `games` channel.

5. Frontend updates instantly

Your React component's Realtime callback receives the payload. It updates local state. React re-renders the game card. The user sees “98–95” without refreshing, without polling, without any action on their part.

User Flow



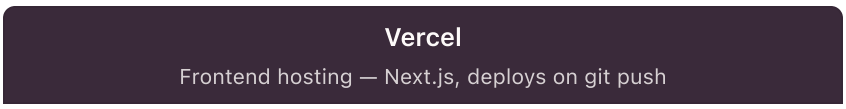
Users create an account (Supabase Auth), select their favorite NBA teams (stored in the `favorites` table with RLS), and see a scoreboard filtered to their teams. The scoreboard updates in real time via Supabase Realtime. The user never interacts with the worker or ESPN — the data just appears.

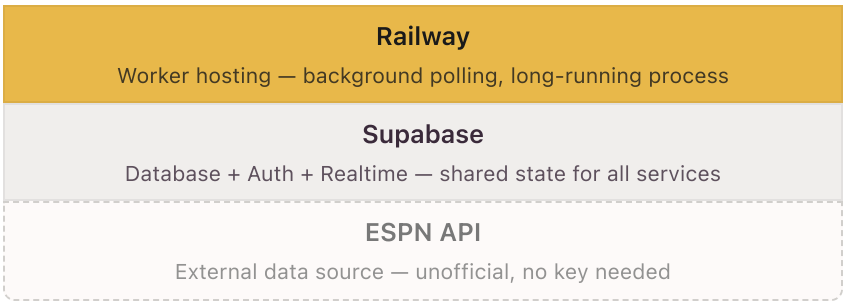
Mental Model Check

If you can answer these five questions, you understand the architecture:

> 1. **Why doesn't the browser call ESPN directly?**
> CORS blocks it. It doesn't scale (500 users = 500 calls). There's no persistence. The worker solves all three problems.
> 2. **What does the worker do, and why is it separate from the frontend?**
> Polls ESPN every 30 seconds and writes to Supabase. Separate because it has different runtime needs (runs forever), different scaling needs (one instance), and should keep running if the frontend goes down.
> 3. **What's the difference between pulling and pushing data?**
> Pulling = the client asks repeatedly. Pushing = the database notifies when data changes. Realtime uses push — no wasted requests, instant updates.
> 4. **Why does the worker use the service role key?**
> It's not a user. It has no auth token. RLS would block it with the anon key. Service role bypasses RLS because the worker is trusted infrastructure.
> 5. **Why upsert instead of insert?**
> The same game gets updated many times. Upsert creates the row if new, updates it if it exists. Insert would fail on duplicates.

The Stack





Each layer builds on the one below it. The database is the connective tissue between all services.

11 Quick Reference

ESPN API Endpoint

```
https://site.api.espn.com/apis/site/v2/sports/basketball/nba/scoreboard

# Other sports (same pattern):
# NFL:  .../football/nfl/scoreboard
# MLB:  .../baseball/mlb/scoreboard
# MLS:  .../soccer/usa.1/scoreboard
```

Supabase MCP Setup

```
claude mcp add --transport http supabase https://mcp.supabase.com/mcp
```

Railway Deploy Flow

Step	Action
1	Sign up at **railway.app** (no credit card needed, $5 trial)
2	Create a new project
3	Connect your GitHub repo
4	Set the root directory to `apps/worker`
5	Add environment variables: `SUPABASE_URL`, `SUPABASE_SERVICE_ROLE_KEY`
6	Push to GitHub — Railway auto-deploys

Key Environment Variables by Platform

Platform	Variable	Key Type
Vercel	`NEXT_PUBLIC_SUPABASE_URL`	Public — database URL
Vercel	`NEXT_PUBLIC_SUPABASE_ANON_KEY`	Public — respects RLS
Railway	`SUPABASE_URL`	Server-only — database URL
Railway	`SUPABASE_SERVICE_ROLE_KEY`	Server-only — bypasses RLS

Supabase Realtime Subscribe Pattern

```
const channel = supabase
  .channel('games')
  .on('postgres_changes',
    { event: '*', schema: 'public', table: 'games' },
    payload => {
      // payload.new contains the updated row
      updateGame(payload.new)
    })
  .subscribe()

// Don't forget: enable Replication for the table in the Supabase dashboard!
```

Codex CLI Install

```
npm i -g @openai/codex
codex  # launch Codex CLI
```

Key Vocabulary

System Multiple independent services communicating through shared infrastructure. More than just an app.

Background worker A long-running process that runs on its own schedule, without user triggers. Polling, processing, syncing.

Polling Fetching data at regular intervals. The worker polls ESPN; this is different from the frontend polling the database.

Realtime Supabase feature that pushes database changes to connected clients via WebSockets. Subscribe, don't poll.

WebSocket A persistent, two-way connection between browser and server. Unlike HTTP (request-response), data flows continuously.

Upsert Insert + update. Creates a row if new, updates it if it already exists. Uses `onConflict` to match rows.

Monorepo One Git repository containing multiple projects/services. Shared history, atomic changes, unified context.

Railway Cloud platform for deploying any process — workers, bots, servers. Not limited to web frameworks.

Service role key Supabase secret key that bypasses RLS. For trusted server-side code only. Never in the browser.

Anon key Supabase public key that respects RLS. Safe for browser use. Paired with user auth tokens.

CORS Cross-Origin Resource Sharing. Browser policy that blocks requests to different domains. Server-to-server is not affected.

Unofficial API An endpoint built for internal use (like ESPN's own website), used by third parties without official support.

Codex CLI OpenAI's terminal-based coding agent. Same workflow pattern as Claude Code, different model.

AGENTS.md	Open standard project context file (Linux Foundation). Read by Codex, Copilot CLI, Gemini CLI, and others.
Loose coupling	Services that communicate through shared data, not direct calls. The worker and frontend don't know about each other.

What's Next

Week 5: Project Kickoff. No more assignments. You choose what to build and spend the next five weeks shipping it. The architecture patterns you've learned — auth, databases, real-time data, background workers, multi-service systems — are your building blocks. The only question left is: what do you want to build with them?